

DevOps Engineer – Technical Assignment

Company: Oceans Across Pvt. Ltd.

Time Limit: 24–48 Hours

Submission: Share a GitHub repository link with all code, configuration files, architecture diagram, and a README covering your decisions and setup steps.

Context

You are joining the infrastructure team at Oceans Across, where we are building an end-to-end payroll platform for the UK market. The platform handles highly sensitive data — employee bank account details, social security numbers, payroll records, and employer information. It serves three distinct user types through separate portals: Companies, Bureaus, and Employees, each with strictly isolated access.

This assignment simulates the kind of infrastructure, security, and deployment work you will own in this role.

Task 1 — AWS Infrastructure Setup

Design and provision the core AWS infrastructure for the payroll platform using Terraform or CloudFormation. Your setup should include:

- A VPC with public and private subnets across at least two availability zones
- EC2 instances (t2.micro or t3.micro) for running backend services — one per tenant type (Companies, Bureaus, Employees) to reflect isolation at the compute layer
- RDS (PostgreSQL, db.t3.micro) in a private subnet for the primary database
- S3 bucket for storing documents and payroll reports, with versioning enabled
- IAM roles scoped per tenant to enforce access boundaries at the AWS resource level
- Security Groups and NACLs configured to isolate traffic between tenant environments

Provide the infrastructure-as-code files and a clear architecture diagram. You do not need to deploy live — a well-structured plan with working IaC is sufficient.

Task 2 — Multi-Tenancy Architecture

The platform serves three completely separate user types — Companies, Bureaus, and Employees — each with their own portal and data scope. A Company should never be able to see Bureau or Employee data outside its own records. A Bureau should only access the clients assigned to it. An Employee should only ever see their own payroll data.

Design and document a multi-tenancy model for this platform. Your submission should cover:

2a. Tenant Isolation Strategy

- Choose and justify a tenancy model: shared database with tenant_id scoping, schema-per-tenant, or database-per-tenant — explain your reasoning given the sensitivity of payroll data
- Show how tenant context is established at login and propagated securely through the request lifecycle

- Demonstrate how a query or API call is guaranteed to only return data belonging to the authenticated tenant — no cross-tenant leakage under any condition

2b. Access Boundaries at the Infrastructure Layer

- Define separate IAM roles for each portal type (Company, Bureau, Employee) with access strictly limited to their respective AWS resources
- Show how S3 bucket policies or prefixes enforce that a Company cannot access documents belonging to a Bureau or Employee, and vice versa
- Ensure that even if application-level tenant logic fails, the infrastructure layer acts as a second enforcement boundary

2c. Tenant Onboarding & Offboarding

- Describe how a new Company or Bureau is provisioned in the system — what gets created, what gets scoped, and how it is isolated from day one
- Describe how a tenant is cleanly offboarded — data deletion, access revocation, and audit trail

Task 3 — Security & Access Control

Security is the most critical layer of this platform. Implement and document the following:

3a. IAM & Role-Based Access Control

- Create IAM roles and policies that enforce least-privilege access
- Define separate access boundaries for the three tenant types — no role should have access to resources outside its boundary
- Ensure no hardcoded credentials anywhere in your configuration

3b. Secrets Management

- Use AWS Secrets Manager or SSM Parameter Store to manage sensitive values (DB credentials, API keys)
- Show how secrets are injected into the application at runtime without being exposed in code or logs

3c. Encryption

- Enable encryption at rest for RDS and S3
- Configure SSL/TLS for all services exposed over the network
- Document how data in transit is protected between services

3d. Network Security

- Configure Security Groups and NACLs to restrict traffic strictly to what is required
- Ensure the database and internal services are not publicly accessible
- Document how you would prevent one tenant's traffic from reaching another tenant's compute or data layer

Task 4 — CI/CD Pipeline

Set up a GitHub Actions pipeline that does the following:

1. Runs on every push to the main branch

2. Builds and tests a simple Dockerised application (can be a placeholder/hello-world service)
3. Deploys to an EC2 instance using SSH or AWS Systems Manager (SSM) — no paid container registry required
4. Handles environment-specific configurations (e.g., dev vs production) without exposing secrets in the pipeline YAML

The pipeline should be structured so that multiple teams (frontend, backend, AI) can independently trigger deployments to their respective services without interfering with each other.

Task 5 — Monitoring & Incident Readiness

Set up basic observability for the infrastructure:

- Configure CloudWatch alarms for EC2 CPU utilisation and RDS connection thresholds
- Set up CloudWatch log groups for application and infrastructure logs with a sensible retention policy
- Define at least one SNS alert that would notify the team of a critical failure

Additionally, write a brief incident response runbook (one page is enough) covering: how you would detect, investigate, and recover from a database being accidentally made publicly accessible.

Task 6 — UK Compliance Considerations

In a short written section (within your README), answer the following:

1. What AWS-native controls would you put in place to stay compliant with UK GDPR when storing employee PII and bank data?
2. How would you ensure data residency within the UK/EU region?
3. How would you handle a scenario where an employee's data needs to be permanently deleted across all services (right to erasure)?

What to Submit

- GitHub repository (public or shared access) containing all IaC files, pipeline configuration, Dockerfiles, and scripts
- Architecture diagram (image or PDF) showing the full AWS setup
- README with: setup instructions, decisions made, trade-offs considered, and answers to Task 6
- Incident response runbook (can be included in the README or as a separate file)

AI Usage Log (Mandatory)

Use of AI tools (ChatGPT, Claude, Copilot, etc.) is permitted and expected. However, you must submit a complete log of your AI usage as part of your submission. This is not optional.

The log should be included as a separate file (e.g., ai_log.md) in your repository and must cover:

- Every prompt you used, in the exact form you typed it
- The AI's response or the key output you took from it

- What you changed, adapted, or rejected from the AI's suggestion — and why
- Any follow-up prompts used to refine or correct an earlier response

We are not penalising AI use — we are assessing how you use it. A candidate who uses AI thoughtfully and documents it clearly will be viewed more favourably than one who avoids it entirely or uses it without reflection.

Important Notes

You are not expected to provision live AWS resources if you do not have access to an account. Well-structured Terraform/CloudFormation code, a clear architecture diagram, and a detailed README will be evaluated on equal footing with a live deployment. What matters is that your decisions are deliberate, well-reasoned, and security-first.

If you do choose to deploy on AWS, the following are mandatory:

- Use only AWS Free Tier eligible services throughout the assignment. Any service or configuration that falls outside the free tier must not be used. The free-tier-safe services for this assignment are: EC2 (t2.micro / t3.micro), RDS (db.t3.micro), S3, IAM, CloudWatch, SNS, SSM Parameter Store, and Secrets Manager.
- Immediately stop or terminate all EC2 and RDS instances once you have completed the assignment and captured any screenshots or outputs needed for your submission. Do not leave instances running after submission.
- Double-check that no resources are left in a running state after submission. You are solely responsible for any charges incurred on your AWS account. Oceans Across will not be liable for any costs arising from this assignment.